

H0003 — Editar torneo

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

- Épica: Configuración inicial
- 3 puntos
- KANO: Performance
- Rol: Coordinador

1. Qué se implementó

El **Coordinador** puede editar los datos de un torneo existente (nombre, fecha, lugar y flyer). Se agregó el endpoint `PUT`, el caso de uso CQRS con su regla de negocio, y en el frontend se **reutilizó el formulario de alta** para editar, con precarga de datos.

2. Backend (Clean Architecture + CQRS)

Archivo	Rol
<code>Commands/UpdateTorneo/UpdateTorneoCommand.cs</code>	Comando CQRS de edición (Id + datos).
<code>.../UpdateTorneoCommandHandler.cs</code>	Busca el torneo, aplica la regla de negocio y persiste.
<code>.../UpdateTorneoCommandValidator.cs</code>	Id presente, nombre/lugar requeridos, fecha futura.
<code>API/Endpoints/Torneos/UpdateTorneoEndpoint.cs</code>	<code>PUT /api/v1/torneos/{id}</code> , <code>Roles("Coordinador")</code> .
<code>.../UpdateTorneoRequest.cs + Validator</code>	Bind del <code>Id</code> desde la ruta + validación en el borde HTTP.

No hubo entidad ni migración nuevas: se reutilizó `Torneo` y `ITorneoRepository.UpdateAsync` (ya existían). Mapeo de salida con Mapster.

Regla de negocio. Un torneo en estado `Finalizado` es de solo lectura: el handler lanza `ConflictException` (→ HTTP 409) si se intenta editar. Si el torneo no existe, `NotFoundException` (→ 404).

Flujo de edición

```
PUT /torneos/{id} (rol Coordinador)
-> UpdateTorneoRequestValidator (valida el request)
-> UpdateTorneoEndpoint (arma el Command y lo envia con MediatR)
-> ValidationBehavior (pipeline: valida el Command)
-> UpdateTorneoCommandHandler (404 si no existe; 409 si Finalizado; aplica cambios)
-> ITorneoRepository.UpdateAsync (EF Core -> PostgreSQL)
-> 200 OK + TorneoResponse
```

3. Frontend (React 19 + TanStack Query)

Archivo	Rol
---------	-----

<code>types/index.ts</code>	<code>UpdateTorneoInput</code> (misma forma que el alta).
<code>lib/api.ts</code>	<code>torneos.editar(id, data)</code> → PUT.
<code>hooks/useTorneos.ts</code>	<code>useEditarTorneo(id)</code> : mutación que invalida la lista y el detalle en <code>onSuccess</code> .
<code>components/torneos/TorneoForm.tsx</code>	Refactor reusable: sin <code>torneo</code> crea; con <code>torneo</code> precarga y edita.
<code>routes/torneos/\$torneoid/editar.tsx</code>	Página de edición: carga el torneo (loading/error) y monta el form precargado.
<code>components/torneos/TorneoCard.tsx</code>	Enlace “Editar” (solo Coordinador y si el torneo no está Finalizado).
<code>App.tsx</code>	Ruta <code>/torneos/:torneoId/editar</code> protegida con <code>RoleGuard(['Coordinador'])</code> .

4. Decisiones de diseño

Un solo formulario para crear y editar. En vez de duplicar `TorneoForm` , recibe un `torneo?` opcional: define el modo, los valores por defecto, el hook (`useCrearTorneo` vs `useEditarTorneo`), los textos del botón y los mensajes de éxito/error. Menos código y una sola fuente de validación (Zod).

Coherencia UI ↔ regla de negocio. El botón “Editar” se oculta cuando el torneo está Finalizado, reflejando en la UI la misma restricción que el backend garantiza con el 409. El backend sigue siendo la autoridad: la UI es solo una ayuda.

Invalidación de caché. La mutación invalida `torneoKeys.all` (lista) y `torneoKeys.detail(id)` para que tanto el listado como cualquier vista de detalle reflejen el cambio.

5. Pruebas

- `UpdateTorneoCommandHandlerTests` — happy path (actualiza y devuelve), `NotFoundException` si no existe, `ConflictException` si está Finalizado (y no persiste).
- `UpdateTorneoCommandValidatorTests` — Id vacío, nombre vacío y fecha pasada fallan; comando válido pasa.

✓ Backend compila 0 errores · **38/38 tests** en verde (+7).

✓ Frontend `tsc + vite build` sin errores.

Nota: la verificación end-to-end del PUT contra la API quedó pendiente porque el entorno Docker/backend estaba detenido al momento de implementar. Al levantar el entorno, el flujo (editar desde la card → guardar → ver el cambio) queda listo para probar.

6. Checklist

Ítem	Estado
------	--------

Command + Handler + Validator (edición)	✓
Endpoint PUT con Roles("Coordinador")	✓
Regla de negocio: Finalizado = solo lectura (409)	✓
Mapeo Mapster · repositorio reutilizado	✓
Tests (happy, NotFound, Conflict, validaciones)	✓
Types + api.editar + hook useEditarTorneo	✓
TorneoForm reutilizable (crear/editar) + precarga	✓
Ruta editar protegida con RoleGuard	✓
Enlace "Editar" (Coordinador, oculto si Finalizado)	✓
Comentarios XML <code><summary></code> en el código nuevo	✓
Verificación E2E del PUT	pendiente (entorno caído)